

DSC3103 Pipeline Presentation Guide

1. What the pipeline does

The pipeline reads two related datasets: commodity prices from Source A and daily rainfall from Source B. It cleans the price data, joins the clean prices to rainfall using `date` and `market`, and stores the merged result as a Parquet file. The complete command is:

```
```bash
python -m src.run_pipeline
```
```

The `-m` option runs `src.run\_pipeline` as a package module. This matters because imports such as `from src.ingest...` need the project root on Python's import path.

2. The complete file

The current `src/run\_pipeline.py` is:

```
```python
from src.common.logging_setup import get_logger
from src.ingest.source_a import ingest_source_a
from src.ingest.source_b import ingest_source_b
from src.transform.clean import clean_data
from src.transform.merge import merge_data

PATH = "data/processed/merged_prices_with_rain.parquet"
logger = get_logger()

def run_pipe():
 logger.info("=== Pipeline run started ===")

 logger.info("Stage 1: ingesting source A")
 source_a = ingest_source_a()
 logger.info("Source A ingested rows=%d", len(source_a))

 logger.info("Stage 2: ingesting source B")
 source_b = ingest_source_b()
 logger.info("Source B ingested rows=%d", len(source_b))

 logger.info("Stage 3: cleaning data")
 clean_prices, decisions = clean_data()
 logger.info(
 "Source A cleaned rows_in=%d rows_out=%d decisions=%d",
 len(source_a),
 len(clean_prices),
 len(decisions),
)
```

```

)

logger.info("Stage 4: merging prices and rainfall")
merged = merge_data(clean_prices, source_b)
logger.info("Merge complete rows=%d", len(merged))

logger.info("Stage 5: Saving the merged file")
merged.to_parquet(PATH, index=False)
logger.info(f"Stage 55: Saving complete: saved {len(merged)} rows")
logger.info("=== Pipeline run completed ===")

return merged

if __name__ == "__main__":
 run_pipe()
...

```

### 3. Line-by-line explanation

#### Imports

``from src.common.logging_setup import get_logger`` imports the project's logging factory. Logging records what happened during a run and writes messages to the console and log file.

``from src.ingest.source_a import ingest_source_a`` imports the Source A loader. It reads ``data/raw/prices.csv`` and returns a pandas DataFrame.

``from src.ingest.source_b import ingest_source_b`` imports the Source B loader. It reads the cached rainfall CSV, or calls the configured rainfall API if the cache does not exist.

``from src.transform.clean import clean_data`` imports the cleaning stage. It rejects bad prices, duplicate rows, duplicate IDs and invalid dates, while imputing missing markets and normalizing text.

``from src.transform.merge import merge_data`` imports the join stage. Keeping this in its own module makes the join testable independently from the full pipeline.

#### Configuration and logger

``PATH = ...`` gives the processed output a stable destination. The output is a Parquet file because Parquet preserves column types and is efficient for analytical data.

``logger = get_logger()`` creates one configured logger for this module. Every stage can write an informative message without using scattered ``print()`` statements.

#### Function boundary

``def run_pipe():`` defines the one entry point for the workflow. A function boundary makes the pipeline

callable from tests or another Python module.

``logger.info("=== Pipeline run started ===")`` marks the beginning of a run in the log.

## Stage 1: Source A

``logger.info("Stage 1: ingesting source A")`` records the stage name before work begins.

``source_a = ingest_source_a()`` calls the Source A loader. The returned value is a DataFrame containing columns such as ``record_id``, ``date``, ``market``, ``commodity`` and ``price``.

``len(source_a)`` counts rows. In the current data this is 1,010 rows before cleaning.

## Stage 2: Source B

``source_b = ingest_source_b()`` loads rainfall data with ``date``, ``rainfall_mm`` and ``market`` columns.

The shared keys are ``date`` and ``market``. This is why the datasets are related: a price observation at a market on a date can be paired with rainfall observed at that same market and date.

## Stage 3: Cleaning

``clean_prices, decisions = clean_data()`` returns two values. ``clean_prices`` is the usable DataFrame.

``decisions`` is a list describing actions such as rejected rows and imputations.

``len(source_a)`` is the input count, while ``len(clean_prices)`` is the output count. The current run changes 1,010 source rows into 918 clean rows.

``len(decisions)`` counts the recorded cleaning decisions. Logging both input and output counts makes data loss visible to a reviewer.

## Stage 4: Merge

``merged = merge_data(clean_prices, source_b)`` performs a left join. Every clean price row remains in the result; rainfall is added where ``date`` and ``market`` match.

The merge uses ``validate="many_to_one"``. This is a defensive check: each rainfall key should identify at most one rainfall row. If Source B unexpectedly contains duplicate keys, the pipeline fails instead of silently multiplying price rows.

``len(merged)`` reports the final row count. A correct left join should normally keep the 918 cleaned price rows.

## Stage 5: Storage

``merged.to_parquet(PATH, index=False)`` writes the final DataFrame to the configured output path.

``index=False`` prevents pandas' internal row index from becoming an unwanted data column.

Running the command twice overwrites the same file. This is the pipeline's idempotency mechanism: the second run does not append another copy of the rows.

``return merged`` makes the final DataFrame available to callers and tests.

## Main guard

``if __name__ == "__main__":`` checks whether this file was executed directly as a module rather than imported.

``run_pipe()`` starts the pipeline only for direct execution. Without this guard, importing the module would unexpectedly run the whole data process.

## 4. Data-flow example

```
```text
prices.csv (1,010 rows)
  |
  v
Source A ingestion
  |
  v
Cleaning and validation (918 rows)
  |          rainfall.csv (728 rows)
  |          |
  +-----+ date + market ---+
          |
          v
merged_prices_with_rain.parquet (918 rows)
```
```

Example join:

```
```text
Price row:  date=2020-01-26, market=Mukono, price=1643
Rain row:   date=2020-01-26, market=Mukono, rainfall_mm=0.8
Merged row: price=1643, rainfall_mm=0.8
```
```

If a price row has no matching rainfall observation, the left join keeps the price row and sets ``rainfall_mm`` to missing (``NaN``). That is preferable to silently deleting a valid price observation.

## 5. Questions a panel may ask

**\*\*Why use two sources?\*\*** The price and rainfall datasets describe the same market-date context, so rainfall can help analyze environmental effects on commodity prices.

**\*\*Why clean before merging?\*\*** Invalid dates, prices and duplicate IDs would make the join unreliable and

could create incorrect analysis results.

**\*\*Why use a left join?\*\*** Prices are the primary observations. We want to retain every valid price row even when rainfall is unavailable.

**\*\*Why use Parquet?\*\*** It is compact, preserves types better than CSV, and is designed for analytical workflows.

**\*\*How is the pipeline idempotent?\*\*** The output file is overwritten on each run rather than appended to.

**\*\*What fails fast?\*\*** A missing required source file raises an informative `FileNotFoundError`; an invalid join cardinality raises a merge validation error.

**\*\*How do you run it?\*\*** From the repository root: `python -m src.run_pipeline``.

## 6. Current run evidence

The verified run produced:

- Source A: 1,010 rows
- Source B: 728 rows
- Cleaned prices: 918 rows
- Merged output: 918 rows
- Output: `data/processed/merged_prices_with_rain.parquet``